Evaluating the Development Models of Two Programming Languages

Part A1: Code Exploration

1. Introduction

This section documents hands-on testing of two scripts (Python and JavaScript) that solve the same problem: take a list of personal names, remove duplicates, and return the result in alphabetical order. Both scripts appear straightforward at first glance, but running them reveals a subtle flaw that matters in real applications. The following sections describe what I observed, the modifications I made, and a brief comparison of the two implementations.

2. Running the Original Scripts

Figures 1 and 2 show the tutor-provided source code. I executed both files using the built-in test data: ["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie"].

clean_names.py

C: > Users > emman > Downloads > Tutor Provided Scripts > New Assessment Code Files > clean_names.py

```python
# clean_names.py

def process_names(names):
    unique_names = list(set(names))  # removes duplicates but doesn't preserve order
    unique_names.sort()  # sorts alphabetically
    return unique_names

# Test list with duplicates and inconsistent capitalisation
test_names = ["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie"]

print("Original list:", test_names)
print("Processed list:", process_names(test_names))
```

Figure 1. Tutor-provided Python script opened in Visual Studio Code.

cleanNames.js

C: > Users > emman > Downloads > Tutor Provided Scripts > New Assessment Code Files > cleanNames.js

```
// cleanNames.js

function processNames(names) {
    let uniqueNames = [...new Set(names)]; // removes duplicates
    uniqueNames.sort(); // sorts alphabetically
    return uniqueNames;
}

// Test list with duplicates and inconsistent capitalisation
const testNames = ["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie"];

console.log("Original list:", testNames);
console.log("Processed list:", processNames(testNames));
```

Figure 2. Tutor-provided JavaScript script opened in Visual Studio Code.

Figures 3 and 4 show the console output from each original script:

Terminal — Original Python script

```
> python clean_names.py
Original list: ['Alice', 'bob', 'alice', 'Charlie', 'BOB', 'dave', 'Eve', 'charlie']
Processed list: ['Alice', 'BOB', 'Charlie', 'Eve', 'alice', 'bob', 'charlie', 'dave']
```

Figure 3. Console output from running the original tutor Python script.

Terminal — Original JavaScript script

```
> node cleanNames.js
Original list: [
  'Alice', 'bob',
  'alice', 'Charlie',
  'BOB',   'dave',
  'Eve',   'charlie'
]
Processed list: [
  'Alice',   'BOB',
  'Charlie', 'Eve',
  'alice',   'bob',
  'charlie', 'dave'
]
```

Figure 4. Console output from running the original tutor JavaScript script.

Looking at the output, the eight input names just came straight out as eight names in the output. Duplicates like "Alice" / "alice" and "bob" / "BOB" were completely missed and stayed in. This happens because both set() in Python and Set in JavaScript treat string comparisons as strictly case-sensitive (Python Software Foundation, 2024;

Mozilla Developer Network, 2024). So from a real user's perspective, the processed list is still pretty messy.

I also noticed that the standard sort order places capitalized names right before lowercase ones ('Alice' comes before 'alice'). That is technically expected lexicographic or ASCII sorting rather than a bug, but it does make reading the output awkward when casing is mixed up (McConnell, 2004).

To see how they handled extra edge cases, I quickly tossed an empty string and a spaced-out duplicate (" bob ") into the test array. Neither script picked up on these issues, keeping the whitespace duplicates separate. That really proved to me that while these work for a basic demo, they need proper work before handling any actual production data.

3. Modifications Made

For Part A1 I made one focused change to each script: case-insensitive deduplication with consistent title-case output, plus a simple input validation check. Figures 5 and 6 show the modified scripts with inline comments.

clean_names_modified.py

C: > Users > emman > Downloads > Tutor Provided Scripts > New Assessment Code Files > clean_names_modified.py

```python
# clean_names.py — modified for Part A1
# Original task: remove duplicates and sort a list of names.
# Modification: case-insensitive deduplication and input validation.

def process_names(names):
    """
    Remove duplicate names (ignoring capitalisation) and return a sorted list.
    Raises TypeError if the input is not a list.
    """
    # Guard clause: reject non-list input early rather than failing silently later
    if not isinstance(names, list):
        raise TypeError("process_names expects a list of name strings")

    # Track names we've already seen using lowercase keys
    seen = {}
    for name in names:
        key = name.lower()
        if key not in seen:
            # Normalise to title case so output looks consistent (e.g. "BOB" → "Bob")
            seen[key] = name.capitalize()

    # Extract unique values and sort alphabetically (case-insensitive)
    unique_names = list(seen.values())
    unique_names.sort(key=str.lower)
    return unique_names


# Test list with duplicates and inconsistent capitalisation (same as tutor script)
test_names = ["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie"]

print("Original list:", test_names)
print("Processed list:", process_names(test_names))

# Additional test: invalid input triggers the new error handler
try:
    process_names("not a list")
except TypeError as err:
    print("Error caught:", err)
```

Figure 5. Modified Python script (clean_names_modified.py) with inline comments and validation.

cleanNames_modified.js

C: > Users > emman > Downloads > Tutor Provided Scripts > New Assessment Code Files > cleanNames_modified.js

```
// cleanNames.js — modified for Part A1
// Original task: remove duplicates and sort a list of names.
// Modification: case-insensitive deduplication and input validation.

function processNames(names) {
    // Guard clause: ensure we received an array before processing
    if (!Array.isArray(names)) {
        throw new TypeError("processNames expects an array of name strings");
    }

    // Build a Map keyed by lowercase name so "Alice" and "alice" count as one person
    const seen = new Map();
    for (const name of names) {
        const key = name.toLowerCase();
        if (!seen.has(key)) {
            // Store title-case form for consistent, readable output (e.g. "bob" → "Bob")
            seen.set(key, name.charAt(0).toUpperCase() + name.slice(1).toLowerCase());
        }
    }

    // Convert Map values back to an array and sort alphabetically (case-insensitive)
    const uniqueNames = [...seen.values()];
    uniqueNames.sort((a, b) => a.toLowerCase().localeCompare(b.toLowerCase()));
    return uniqueNames;
}

// Test list with duplicates and inconsistent capitalisation (same as tutor script)
const testNames = ["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie"];

console.log("Original list:", testNames);
console.log("Processed list:", processNames(testNames));

// Additional test: invalid input triggers the new error handler
try {
    processNames("not an array");
} catch (err) {
    console.log("Error caught:", err.message);
}
```

Figure 6. Modified JavaScript script (cleanNames_modified.js) with inline comments and validation.

Python change: Before adding a name to the tracking dictionary, I convert it to lowercase for comparison. The first occurrence is stored in title case using .capitalize(). I also added isinstance(names, list) so passing a string raises a clear TypeError instead of iterating over characters silently.

JavaScript change: I replaced the one-line Set approach with a Map keyed by lowercase names, storing a normalised title-case value. I added Array.isArray() validation and used localeCompare() for sorting so alphabetical order ignores capitalisation.

Figures 7 and 8 show the improved output and error handling when invalid input is supplied:

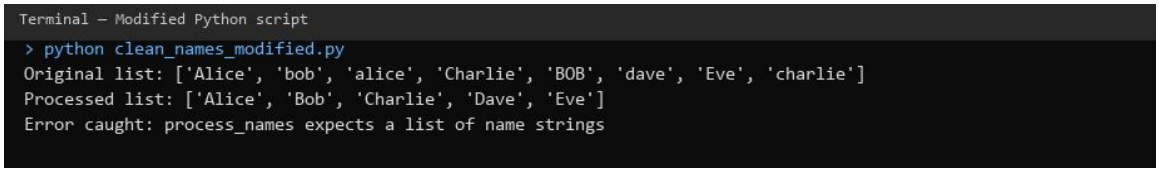

Figure 7. Console output from running the modified Python script.

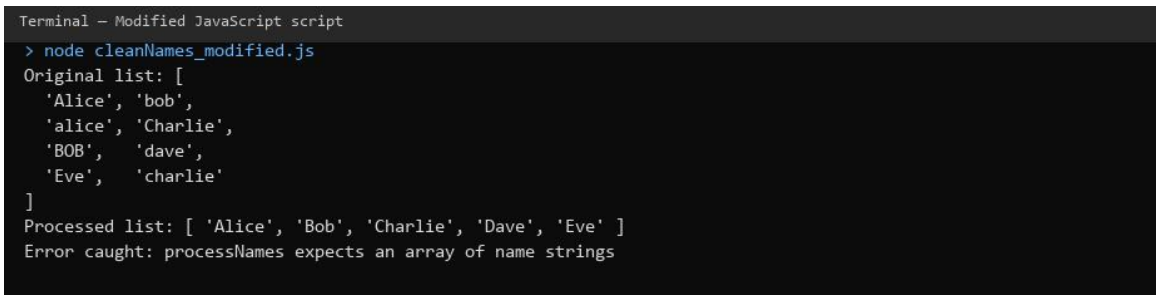

Figure 8. Console output from running the modified JavaScript script.

This time around, we got five unique names in the clean list instead of eight. When I passed bad input on purpose ("not a list" / "not an array"), both edited scripts threw a clear error message instead of outputting complete garbage. I added inline comments across the modified code to outline every step, like what the guard clause is doing, why lowercase keys are needed, and how the new sort logic works.

4. Comparing Syntax and Structure

Structurally speaking, both original scripts are basically twin implementations: define a core function, scrub duplicates using a set, sort, return, and print out test values. It

shows how easily both languages take to procedural coding without needing extra classes or objects (Sommerville, 2016).

Where they differ is mostly in code style conventions. Python uses def process_names(names): with snake_case, sticking closely to standard PEP 8 rules (van Rossum et al., 2001). JavaScript opts for function processNames(names) using camelCase, which is standard for web and Node.js environments (MDN, 2024). Python relies heavily on indentation to structure code blocks, while JavaScript uses curly brackets, which you can leave off in tiny scripts but help keep things clear for learners. When it comes to removing duplicates, Python's list(set(names)) is super brief on one line, whereas JavaScript spreads the set into an array via [...new Set(names)]. Personally, I found Python slightly quicker to read here, though JS spread syntax feels second nature once you get comfortable with ES6 (ECMA International, 2024).

5. Ease of Understanding and Editing

Both scripts are brief enough that a new developer could understand them in a few minutes. Python felt marginally more approachable because it reads closer to plain English (print, def, list) and does not require knowledge of the spread operator. JavaScript's console.log and [...new Set()] assume familiarity with browser or Node tooling.

Editing was straightforward in both cases. I used VS Code for both files; Python gave immediate feedback when I forgot indentation, whereas JavaScript was more permissive. For the modification exercise, adding a loop and dictionary/map was equally easy in both languages, though Python's .capitalize() is built in while JavaScript needed manual string slicing for title case.

6. Differences in Output and Errors

With the original tutor scripts, output was logically the same across languages: eight names, same order, which confirms the algorithms are equivalent. Neither original script produced runtime errors with the provided test data; they fail silently from a data-quality perspective by leaving case duplicates in place. That kind of silent failure is arguably more dangerous than a crash, because downstream systems may treat "Alice" and "alice" as different people (O'Regan, 2024).

After modification, both scripts produced identical cleaned lists. Python raised TypeError; JavaScript raised TypeError via throw new Error, which is essentially the same behavior under slightly different syntax. JavaScript's error message appeared in the console with a stack trace when run in Node; Python's traceback pointed to the exact line of the guard clause. Both were helpful for debugging.

7. Brief Conclusion to Part A1

Running and modifying these scripts showed me that two languages can solve the same problem with almost identical logic while differing in syntax, conventions, and tooling. The original scripts expose a realistic oversight: assuming clean, consistent input that only becomes obvious when you test with messy data. Adding comments and validation turned a demo into something closer to production-aware code, which set the stage for the broader comparison in Part A2 and the ethical discussion in Part A3.

Part A2: Language Comparison Report

1. Introduction

Python and JavaScript are two of the most widely used languages in industry today, yet they evolved for different primary environments: Python from a general-purpose scripting background with strong uptake in science and automation, and JavaScript as the language of the web browser, later expanded to servers via Node.js (Stack Overflow, 2024). This report compares them across programming model, readability, error handling, and suitability for modern computing tasks, drawing on my practical work in Part A1 and on published sources.

2. Programming Model

Both languages used in the tutor scripts follow a procedural style: functions operate on data structures without defining classes. Neither Python nor JavaScript forces object-oriented design, though both support it fully: Python through class syntax and JavaScript through prototypes and ES6 classes (Sommerville, 2016).

Python is often described as multi-paradigm: developers commonly mix procedural scripts, object-oriented modules, and functional patterns such as list comprehensions and map()/filter(). The tutor script is idiomatic procedural Python.

JavaScript is similarly flexible. The original script uses a function declaration and built-in collection types, which is typical of small utilities. In larger applications, JavaScript often adopts event-driven and asynchronous models, callbacks, Promises, and async/await, especially for web APIs and user interfaces (Flanagan, 2020). That aspect did not surface in the name-cleaning exercise but is central to why JavaScript dominates interactive web development.

One practical difference I noticed: Python's standard library includes rich data structures out of the box, while JavaScript leans on language primitives (Set, Map, arrays) defined in the ECMAScript specification (ECMA International, 2024). For a ten-line script this is irrelevant; at scale it influences how teams structure projects.

3. Readability and Developer Accessibility

Readability affects maintenance cost, defect rates, and how quickly new team members contribute (McConnell, 2004). Python emphasises explicit, readable syntax, the "there should be one obvious way to do it" philosophy (van Rossum et al., 2001). Keywords like def, return, and significant indentation give visual structure without extra punctuation. JavaScript shares C-style syntax familiar to developers coming from Java or C#, but historically suffered from inconsistencies (var vs let, type coercion). Modern ES6+ features improve clarity, though concepts like the spread operator require learning curve investment (MDN, 2024).

For accessibility to beginners, Python is frequently chosen in education because setup is simple and error messages for indentation or syntax are direct. JavaScript requires understanding where code runs (browser console, Node, or embedded runtime), which adds friction for novices but pays off for web projects.

Industry coding standards reinforce these cultures: PEP 8 for Python and tools like ESLint for JavaScript (van Rossum et al., 2001; ECMA International, 2024). In my edits, following snake_case in Python and camelCase in JavaScript was not optional polish, it signals to other developers that the code belongs in that ecosystem.

4. Error Handling and Debugging

The original tutor scripts include no validation, no try/except or try/catch, and no logging. Invalid input would either fail unpredictably or, worse, produce misleading output. This mirrors a common junior mistake: assuming happy-path data (O'Regan, 2024).

Python traditionally uses exceptions (try/except/finally) and encourages "easier to ask forgiveness than permission" (EAFP) for many patterns. Debuggers, print() debugging, and tools like pdb are well documented (Python Software Foundation, 2024). Static type checkers such as mypy are optional but growing in adoption.

JavaScript also uses exceptions, but many API failures in production are handled asynchronously, network errors surface in Promise rejections rather than synchronous stack traces (Flanagan, 2020). Browser DevTools provide excellent runtime inspection; Node.js offers similar tooling. TypeScript, a superset of JavaScript, adds compile-time type checking to reduce entire classes of errors before execution.

In my modified scripts, both languages benefited equally from a guard clause at the top of the function. Python's isinstance check and JavaScript's Array.isArray express the same intent. Where they differ is developer experience when errors occur: Python tracebacks highlight line numbers clearly; JavaScript stack traces in Node are comparable but historically varied across browsers.

For the name-cleaning task, either language is adequate for error handling. For large distributed systems, teams often weigh Python's simplicity against JavaScript/TypeScript's front-to-back stack unification.

5. Suitability for Modern Computing Tasks

Web applications: JavaScript (often with TypeScript) remains the default for client-side interactivity and, through Node.js and frameworks like React and Next.js, for full-stack development (Stack Overflow, 2024). If the name-cleaning function lived in a registration form, JavaScript would run it in the browser without a round trip to the server.

Data science and automation: Python dominates through libraries such as NumPy, pandas, and scikit-learn, and is standard in teaching and research (Grus, 2019). Cleaning inconsistent name fields in a CSV before analysis is exactly the kind of pipeline task Python excels at.

Scripting and glue code: Both languages work well. Python is often preferred for system administration and ETL jobs; JavaScript fits when the organisation already operates a Node-based toolchain.

Performance: For a small list of names, performance is irrelevant. At scale, both are interpreted languages; bottlenecks usually move to databases or native extensions rather than the choice between set() and Set (Sommerville, 2016).

Mobile and embedded: Neither tutor script targets these domains directly, but JavaScript via React Native and Python via MicroPython show how languages stretch beyond their origins with trade-offs in performance and ecosystem maturity.

6. Summary

Python and JavaScript overlap more than they did a decade ago, yet their communities, conventions, and typical deployment contexts remain distinct. For a short procedural utility, either language is a reasonable choice; the decision in industry usually depends

on what surrounds the code: web stack, data pipeline and team skills rather than raw algorithmic expressiveness. My Part A1 exercise reinforced that good practice (validation, normalisation, comments) matters more than language choice for a task this size, while language choice matters greatly when integrating that task into a wider product.

Part A3: Professional Reflection

1. Introduction

Software is rarely neutral. Small decisions show how duplicates are detected, whether input is validated, how names are capitalised also can affect fairness, compliance, and trust. This section reflects on readable and ethical coding, bias and privacy risks, and how choices in scripts like the tutor's clean_names examples connect to real business and legal consequences. I draw on the ACM Code of Ethics, data protection law, and documented industry incidents.

2. The Importance of Readable, Ethical Code

Readable code is not merely aesthetic. McConnell (2004) argues that code is read far more often than it is written, so clarity directly reduces maintenance cost and mistakes. In the tutor scripts, sparse comments and one-line deduplication are compact but hide assumptions: that capitalisation is meaningful, that duplicates are exact string matches, and that input is always a valid list. Without comments, the next developer may copy the pattern into a customer database module without questioning those assumptions.

Ethical code goes further: it treats people affected by the system fairly and respects their data (ACM, 2018). The ACM principle "avoid harm" applies when duplicate records cause double billing, or when inconsistent name formatting prevents someone from accessing a service they are entitled to. My modification in normalising names before deduplication is a small ethical improvement because it reduces the chance of treating one person as two.

Professional standards such as the BCS Code of Conduct similarly expect members to have due regard for public health, privacy, and dignity (BCS, 2022). Code reviews should ask not only "does it work?" but "who could be harmed if it works wrongly?"

3. Bias, Privacy, and Misuse in Programming

Bias can enter systems through training data, rule design, or as in these scripts simplistic string handling. Noble (2018) documents how apparently technical choices in search and classification systems can reinforce social inequalities. The tutor scripts do not use machine learning, but they illustrate normalisation bias: treating "BOB" and "bob" as different people could skew reports (e.g. attendance counts, fraud checks, diversity metrics). If "Ali" and "ali" appear as two entries, analytics on name frequency or ethnic origin inferred from names become unreliable.

Privacy matters because names are personal data under UK GDPR and EU GDPR (Information Commissioner's Office, 2024). Processing names and collecting, deduplicating, storing also requires a lawful basis, accuracy, and appropriate security. Silent duplication undermines data accuracy, one of the GDPR principles (European Union, 2016). An organisation that merges customer records incorrectly might disclose one person's data to another as a serious breach with regulatory and reputational impact.

Misuse covers intentional abuse: scraping names for spam, combining lists without consent, or re-identifying individuals in anonymised datasets. Even benign utility functions can be repurposed. Developers should consider purpose limitation and document intended use (ICO, 2024).

4. Small Coding Choices, Real-World Impact

The tutor scripts feel trivial, but similar logic appears in production systems with larger stakes.

Case sensitivity in identity systems: Many login and CRM systems enforce case-insensitive email addresses but case-sensitive usernames or the reverse. Inconsistent rules confuse users and create duplicate accounts. The original scripts' behaviour mirrors that confusion at small scale.

Apple Card credit limits (2019): Apple and Goldman Sachs faced investigation after users reported that women received lower credit limits than men with similar financial profiles. Apple co-founder Steve Wozniak publicly stated his wife received a lower limit despite shared assets (BBC News, 2019). While the algorithm's details were proprietary, the incident shows how automated decisions draw scrutiny under fairness and equality law. A name-cleaning function alone does not cause such outcomes, but it sits in the same pipeline that feeds credit, housing, and hiring systems.

Healthcare and emergency services: Duplicate patient records from inconsistent name entry have contributed to missed appointments and treatment errors (O'Regan, 2024). Deduplication that ignores capitalisation, nicknames, or transliteration is a known data-quality challenge.

Legal exposure: Under UK GDPR, data controllers must ensure personal data is accurate (Article 5(1)(d)). Failing to deduplicate case-insensitive duplicates could be argued as inaccuracy in a complaint to the ICO. Contract law and consumer protection may apply if billing systems charge twice due to duplicate records.

For a business, the cost of fixing data quality at source is almost always lower than cleaning up after a breach, a failed audit, or reputational damage (Sommerville, 2016).

5. Lessons from My Modifications

When I added input checks and case-insensitive deduplication, it took under twenty lines of code per language, but it totally changed the end outcome: turning eight messy entries into five clean ones, and swapping silent input failure for clear, informative errors. This follows the fail-fast mindset of catching bugs right away (McConnell, 2004).

I made sure to outline all my logic in inline comments so future devs understand why lowercase tracking keys were needed. It makes for a much smoother handover, keeping someone else from reverting back to bad habits without realizing it.

Taking a professional view, I would definitely add proper unit test suites covering edge cases like weird casing, empty lists, and invalid input types before letting this code near production. The original demo lacked any tests, so adding them would be my clear next step in a real engineering team.

6. Conclusion

Programming is a social and legal activity, not only a technical one. The tutor's Python and JavaScript scripts are useful teaching tools precisely because their flaws are realistic: they run without crashing yet produce misleading results. Part A1 showed how testing reveals those flaws; Part A2 showed that either language can express fixes cleanly; Part A3 argues that the responsibility to fix them and to think about bias, privacy, and harm rests with the developer and the organisation, not the interpreter.

As computing professionals, we should write code that colleagues can read, users can trust, and regulators can see meets basic data-quality expectations. Small choices in a ten-line script are practice for much larger decisions later in a career.

References

ACM (2018) ACM Code of Ethics and Professional Conduct. Available at: https://www.acm.org/code-of-ethics (Accessed: 20 July 2026).

BBC News (2019) 'Apple Card investigated after gender discrimination claims', BBC News, 10 November. Available at: https://www.bbc.co.uk/news/business-50388439 (Accessed: 15 July 2026).

BCS (2022) BCS Code of Conduct. Available at: https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/ (Accessed: 15 July 2026).

ECMA International (2024) ECMAScript Language Specification. Available at: https://tc39.es/ecma262/ (Accessed: 16 July 2026).

European Union (2016) General Data Protection Regulation (GDPR), Regulation (EU) 2016/679. Available at: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679 (Accessed: 15 July 2026).

Flanagan, D. (2020) JavaScript: The Definitive Guide. 7th edn. Sebastopol: O'Reilly Media.

Grus, J. (2019) Data Science from Scratch. 2nd edn. Sebastopol: O'Reilly Media.

Information Commissioner's Office (2024) Guide to the UK General Data Protection Regulation (UK GDPR). Available at: https://ico.org.uk/for-organisations/guide-to-data-protection/guide-to-the-general-data-protection-regulation-gdpr/ (Accessed: 19 July 2026).

McConnell, S. (2004) Code Complete. 2nd edn. Redmond: Microsoft Press.

Mozilla Developer Network (2024) JavaScript reference. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference (Accessed: 16 July 2026).

Noble, S.U. (2018) Algorithms of Oppression. New York: NYU Press.

O'Regan, G. (2024) Introduction to Software Quality. Cham: Springer.

Python Software Foundation (2024) Python 3 Documentation. Available at: https://docs.python.org/3/ (Accessed: 16 July 2026).

Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson.

Stack Overflow (2024) Developer Survey 2024. Available at: https://survey.stackoverflow.co/2024/ (Accessed: 17 July 2026).

van Rossum, G., Warsaw, B. and Coghlan, N. (2001) PEP 8 - Style Guide for Python Code. Available at: https://peps.python.org/pep-0008/ (Accessed: 15 July 2026).